

# Building a CI/CD Pipeline from Scratch

A complete, step-by-step walkthrough — Python + pytest + GitHub Actions on macOS  
Session date: September 2-3, 2026 | Repository: [github.com/ftk1000/ci-cd-demo](https://github.com/ftk1000/ci-cd-demo) | Local path: `~/CODING/ci-cd-demo`

## The Goal (worked backwards)

**End state:** You push a commit → GitHub automatically runs your tests → if they pass, it builds and deploys an artifact. If they fail, nothing ships.

| Stage     | What we built                                     |
|-----------|---------------------------------------------------|
| 5. CD     | A deploy job that only runs if tests pass         |
| 4. CI     | A GitHub Actions workflow that runs on every push |
| 3. Remote | The repo pushed to GitHub                         |
| 2. Code   | A tiny Python package + pytest tests              |
| 1. Local  | A git repo and the tools to run it                |

**The governing analogy — the kitchen pass-through.** You (the cook) hand off a plate. An automated expediter checks it against the ticket before it ever reaches the customer. Nothing gets served that fails inspection. In our pipeline, `needs: test` is the expediter, and the cook does not get a vote.

## Final Repository Structure

```
ci-cd-demo/
├── .github/
│   └── workflows/
│       └── ci.yml          ← the pipeline (test job + deploy job)
├── src/
│   └── calculator/
│       ├── __init__.py    ← makes it an importable package
│       └── core.py         ← the product
├── tests/
│   └── test_core.py       ← the inspection tickets
├── pyproject.toml         ← how to build & install
├── .gitignore             ← keeps machine-specific junk out of git
├── # Present on disk but NOT committed (ignored):
│   ├── .venv/             ← local virtual environment
│   └── .pytest_cache/     ← pytest's scratch space
```

## Table of Contents

|         |                                 |         |                                   |
|---------|---------------------------------|---------|-----------------------------------|
| Step 1  | Check your tools                | Step 13 | Commit the workflow               |
| Step 2  | Install the GitHub CLI          | Step 14 | Create the remote repo and push   |
| Step 3  | Authenticate the GitHub CLI     | Step 15 | Watch the run                     |
| Step 4  | Initialize the local repository | Step 16 | Confirm tests ran remotely        |
| Step 5  | Write the application code      | Step 17 | Add the deploy job                |
| Step 6  | Write the tests                 | Step 18 | Push and watch the full pipeline  |
| Step 7  | Add the packaging config        | Step 19 | Confirm both jobs ran in sequence |
| Step 8  | Create a venv and install       | Step 20 | Break it on purpose               |
| Step 9  | Run the tests locally           | Step 21 | Push the broken code              |
| Step 10 | Add a .gitignore                | Step 22 | Confirm deploy was skipped        |
| Step 11 | Make your first commit          | Step 23 | Fix the bug                       |
| Step 12 | Write the CI workflow           | Step 24 | Push the fix                      |

# Part I — Local Setup

## Step 1 Check your tools

We need `git` (version control), `python3` (the app + tests), and `gh` (GitHub CLI — makes creating the remote repo a one-liner).

### COMMAND

```
mkdir ci-cd-demo
cd ci-cd-demo/
git --version && python3 --version && gh --version
```

### OUTPUT

```
git version 2.52.0
Python 3.14.6
-bash: gh: command not found
```

Git and Python were fine. Only `gh` was missing, which is expected on a fresh Mac.

## Step 2 Install the GitHub CLI

### COMMAND

```
brew install gh
gh --version
```

### OUTPUT

```
gh version 2.99.0 (2026-09-01)
https://github.com/cli/cli/releases/tag/v2.99.0
```

If `brew` itself had been missing, Homebrew would need installing first.

## Step 3 Authenticate the GitHub CLI

This connects `gh` to your GitHub account so it can create the remote repo later.

### COMMAND

```
gh auth login
```

| Prompt                                         | Answer chosen            |
|------------------------------------------------|--------------------------|
| What account do you want to log into?          | GitHub.com               |
| Preferred protocol for Git operations?         | HTTPS                    |
| Authenticate Git with your GitHub credentials? | Yes                      |
| How would you like to authenticate?            | Login with a web browser |

### OUTPUT

```
! First copy your one-time code: C44B-28DE
Press Enter to open https://github.com/login/device in your browser...
✓ Authentication complete.
- gh config set -h github.com git_protocol https
✓ Configured git protocol
✓ Logged in as ftk1000
```

## Step 4 Initialize the local repository

### COMMAND

```
cd ~/CODING/ci-cd-demo
git init -b main
git status
```

### OUTPUT

```
Initialized empty Git repository in /Users/paulpaul/CODING/ci-cd-demo/.git/
```

```
On branch main
```

```
No commits yet
```

```
nothing to commit (create/copy files and use "git add" to track)
```

**Why `-b main`?** It names the first branch `main` up front. This matters because GitHub Actions workflows are usually configured to trigger on that exact branch name. A mismatch here means your pipeline silently never fires.

## Part II — The Code

### Step 5 Write the application code

CI needs something to actually test. We created a package folder and a tiny module.

#### COMMAND

```
mkdir -p src/calculator
touch src/calculator/__init__.py
cat > src/calculator/core.py << 'EOF'
"""A tiny calculator module – the 'product' our pipeline will test and ship."""

def add(a: float, b: float) -> float:
    return a + b

def subtract(a: float, b: float) -> float:
    return a - b

def divide(a: float, b: float) -> float:
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return a / b
EOF
cat src/calculator/core.py
```

Two notes on what this does:

- `__init__.py` is what makes `calculator` an *importable Python package* rather than just a folder on disk.
- `divide` deliberately raises on zero. That gives us a failure path to test, which is more interesting than three happy-path functions.

**Shell technique — the here doc.** `cat > file << 'EOF' ... EOF` writes everything between the markers into the file. Quoting the marker as `'EOF'` (rather than bare `EOF`) tells bash *not* to expand `$variables` or backticks inside the block — essential when writing code or YAML that contains dollar signs.

### Step 6 Write the tests

#### COMMAND

```
mkdir -p tests
cat > tests/test_core.py << 'EOF'
import pytest

from calculator.core import add, subtract, divide

def test_add():
    assert add(2, 3) == 5
    assert add(-1, 1) == 0

def test_subtract():
    assert subtract(10, 4) == 6

def test_divide():
    assert divide(9, 3) == 3

def test_divide_by_zero_raises():
    with pytest.raises(ValueError):
        divide(1, 0)
EOF
```

Three pytest conventions worth knowing, since pytest discovers tests by *naming pattern*:

- The file must start with `test_`, and so must each function inside it. No registration, no decorators — the name is the registration.
- A bare `assert` is the whole assertion API. No `assertEqual`, no special methods.
- `pytest.raises` *inverts* the logic: the test **passes** only if the error is thrown. That is how you test failure paths.

## Step 7 Add the packaging config

At this point `from calculator.core import ...` would still fail — Python has no idea that `src/calculator` is on the import path. This file fixes that.

### COMMAND

```
cat > pyproject.toml << 'EOF'
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "calculator"
version = "0.1.0"
description = "A tiny calculator used to demonstrate a CI/CD pipeline"
requires-python = ">=3.9"
dependencies = []

[project.optional-dependencies]
dev = ["pytest>=8.0"]

[tool.setuptools.packages.find]
where = ["src"]

[tool.pytest.ini_options]
testpaths = ["tests"]
EOF
```

| Section                                      | What it does                                                                      |
|----------------------------------------------|-----------------------------------------------------------------------------------|
| <code>[build-system]</code>                  | Tells pip which tool to use to build the package (setuptools).                    |
| <code>[project]</code>                       | Name, version, and runtime dependencies (here: none).                             |
| <code>[project.optional-dependencies]</code> | The <code>dev</code> extra holds pytest. Users don't need a test runner; CI does. |
| <code>where = ["src"]</code>                 | The <b>src layout</b> . The key line.                                             |
| <code>[tool.pytest.ini_options]</code>       | Tells pytest where tests live, so bare <code>pytest</code> just works.            |

**Why the src layout matters.** Your source lives in `src/` and gets *installed* rather than picked up by accident from the current working directory. This forces your tests to import the package the same way a real user would. Without it, tests can pass locally purely because the files happen to sit next to each other — and then fail for everyone else.

## Step 8 Create a virtual environment and install

The prompt showed `(base)`, meaning Conda's base environment was active. We made a project-local venv instead so this demo can't pollute or be polluted by the global setup.

### COMMAND

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
```

| Command                                | Effect                                                                                                                  |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <code>python3 -m venv .venv</code>     | Creates an isolated Python in a hidden <code>.venv</code> folder.                                                       |
| <code>source .venv/bin/activate</code> | Activates it. Prompt gains a <code>(.venv)</code> prefix.                                                               |
| <code>pip install -e ".[dev]"</code>   | <b>Editable</b> install + the <code>dev</code> extra. Edits to <code>src/</code> take effect immediately, no reinstall. |

### OUTPUT (TAIL)

```
Obtaining file:///Users/paulpaul/CODING/ci-cd-demo
Installing build dependencies ... done
Checking if build backend supports build_editable ... done
Getting requirements to build editable ... done
Preparing editable metadata (pyproject.toml) ... done
Collecting pytest>=8.0 (from calculator==0.1.0)
Using cached pytest-9.1.1-py3-none-any.whl.metadata (7.6 kB)
...
Building wheels for collected packages: calculator
Building editable for calculator (pyproject.toml) ... done
Created wheel for calculator: filename=calculator-0.1.0-0.editable-py3-none-any.whl size=1304
Successfully built calculator
Installing collected packages: pygments, pluggy, packaging, iniconfig, calculator, pytest
Successfully installed calculator-0.1.0 iniconfig-2.3.0 packaging-26.3 pluggy-1.6.0
pygments-2.21.0 pytest-9.1.1
```

This is the local mirror of what the CI runner will do on GitHub's machine. Same recipe, different kitchen.

## Step 9 Run the tests locally — establish the green baseline

Before automating anything, prove it works by hand. This is the baseline the pipeline has to reproduce.

### COMMAND

```
pytest -v
```

### OUTPUT

```
===== test session starts =====
platform darwin -- Python 3.14.6, pytest-9.1.1, pluggy-1.6.0
cachedir: .pytest_cache
rootdir: /Users/paulpaul/CODING/ci-cd-demo
configfile: pyproject.toml
testpaths: tests
collected 4 items

tests/test_core.py::test_add PASSED [ 25%]
tests/test_core.py::test_subtract PASSED [ 50%]
tests/test_core.py::test_divide PASSED [ 75%]
tests/test_core.py::test_divide_by_zero_raises PASSED [100%]

===== 4 passed in 0.00s =====
```

The `-v` flag prints each test name rather than a row of dots. We use it in CI too, so the logs are readable when something breaks.

## Part III — Version Control

### Step 10 Add a `.gitignore`

The venv and pytest caches are machine-specific junk. If they got committed, CI would pull down *your Mac's* binaries and break.

#### COMMAND

```
cat > .gitignore << 'EOF'
# Virtual environments
.venv/
venv/

# Python bytecode
__pycache__/
*.py[cod]

# Build artifacts
build/
dist/
*.egg-info/

# Test caches
.pytest_cache/
.coverage

# macOS
.DS_Store
EOF
git status --short
```

#### OUTPUT

```
?? .gitignore
?? pyproject.toml
?? src/
?? tests/
```

Exactly four entries. No `.venv`, no `src/calculator.egg-info`. The `git status --short` at the end is the real check — it verifies the ignore rules are doing their job before anything gets committed.

### Step 11 Make your first commit

#### COMMAND

```
git add .
git commit -m "Add calculator package with tests"
git log --oneline
```

#### OUTPUT

```
[main (root-commit) 7a08175] Add calculator package with tests
5 files changed, 74 insertions(+)
create mode 100644 .gitignore
create mode 100644 pyproject.toml
create mode 100644 src/calculator/__init__.py
create mode 100644 src/calculator/core.py
create mode 100644 tests/test_core.py

7a08175 (HEAD -> main) Add calculator package with tests
```

**Why the commit matters for CI.** CI does not test your working directory — it tests **committed** code. The hash `7a08175` is the exact thing GitHub will check out and run. Uncommitted changes are invisible to the pipeline.

A directory listing confirmed `.venv` and `.pytest_cache` existed on disk but stayed out of the commit. The ignore rules held.

## Part IV — Continuous Integration

### Step 12 Write the CI workflow

This is the heart of the demo. GitHub looks for YAML files in a specific hidden directory and runs them automatically.

#### COMMAND

```
mkdir -p .github/workflows
cat > .github/workflows/ci.yml << 'EOF'
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Check out the code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install the package and dev dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -e ".[dev]"

      - name: Run tests
        run: pytest -v

EOF
```

| Block                 | Meaning                                                                           |
|-----------------------|-----------------------------------------------------------------------------------|
| <code>on:</code>      | The trigger — run on any push or PR targeting <code>main</code> .                 |
| <code>jobs:</code>    | A workflow contains one or more jobs. Jobs run in parallel unless told otherwise. |
| <code>runs-on:</code> | A fresh Ubuntu VM, spun up from scratch every single time.                        |
| <code>steps:</code>   | An ordered list. If one fails, the rest of the job stops.                         |
| <code>uses:</code>    | A pre-built action someone else maintains (checkout, Python setup).               |
| <code>run:</code>     | Literal shell commands — note these are the <b>same ones you typed by hand</b> .  |

**The path is exact:** `.github/workflows/`. Misspell any part of it and GitHub silently ignores the file — no error, no warning, just a pipeline that never runs. This is the single most common beginner mistake.

**Why `runs-on: ubuntu-latest` is the point of the whole exercise.** Your tests passed on macOS with Python 3.14. This re-runs them on Linux with Python 3.12. If they only worked because of something peculiar to your laptop, CI is where you find out — not your users.

### Step 13 Commit the workflow

#### COMMAND

```
git add .github/workflows/ci.yml
git commit -m "Add CI workflow"
git log --oneline
```

#### OUTPUT

```
[main 81cb5a8] Add CI workflow
1 file changed, 28 insertions(+)
create mode 100644 .github/workflows/ci.yml
```

```
81cb5a8 (HEAD -> main) Add CI workflow
7a08175 Add calculator package with tests
```

Note we staged the file *by name* this time rather than `git add .` — a small habit worth having, since it forces you to be deliberate about what goes into each commit.

## Step 14 Create the remote repo and push

One command does three things: creates the repo on GitHub, adds it as your `origin` remote, and pushes `main`.

### COMMAND

```
gh repo create ci-cd-demo --public --source=. --remote=origin --push
```

| Flag                         | Why                                                                 |
|------------------------------|---------------------------------------------------------------------|
| <code>--public</code>        | Free unlimited Actions minutes. Private repos have a monthly cap.   |
| <code>--source=.</code>      | Use this existing directory rather than making an empty repo.       |
| <code>--remote=origin</code> | Name the connection <code>origin</code> , the conventional default. |
| <code>--push</code>          | Send <code>main</code> up immediately.                              |

### OUTPUT

```
✓ Created repository ftk1000/ci-cd-demo on github.com
https://github.com/ftk1000/ci-cd-demo
✓ Added remote https://github.com/ftk1000/ci-cd-demo.git
Enumerating objects: 15, done.
Counting objects: 100% (15/15), done.
Writing objects: 100% (15/15), 1.79 KiB | 1.79 MiB/s, done.
To https://github.com/ftk1000/ci-cd-demo.git
* [new branch]      HEAD -> main
branch 'main' set up to track 'origin/main'.
✓ Pushed commits to https://github.com/ftk1000/ci-cd-demo.git
```

**The pipeline fires on this command.** The moment the push lands, GitHub sees `.github/workflows/ci.yml`, matches the `on: push` trigger, and starts a runner. There is no button to click.

## Step 15 Watch the run

### COMMAND

```
gh run list
```

### OUTPUT

| STATUS | TITLE           | WORKFLOW | BRANCH | EVENT | ID          | ELAPSED | AGE                    |
|--------|-----------------|----------|--------|-------|-------------|---------|------------------------|
| ✓      | Add CI workflow | CI       | main   | push  | 33716664512 | 15s     | less than a minute ago |

**Green check on the first try** — 15 seconds on a machine you have never touched.

Related commands: `gh run watch` streams a run's status live (it returns `found no in progress runs to watch` if the run already finished — GitHub is often faster than you can type). `gh run view --log` shows full logs.

## Step 16 Confirm the tests ran remotely

### COMMAND

```
gh run view 33716664512 --log | grep -E "PASSED|passed|platform"
```

**Gotcha:** `gh run view --log | grep ...` without a run ID fails with `run or job ID required when not running interactively`. Piping into `grep` detaches `gh` from the terminal, so it cannot show its interactive picker. Pass the ID explicitly whenever you pipe.

### OUTPUT

```
test Run tests platform linux -- Python 3.12.14, pytest-9.1.1, pluggy-1.6.0
-- /opt/hostedtoolcache/Python/3.12.14/x64/bin/python
test Run tests tests/test_core.py::test_add PASSED [ 25%]
test Run tests tests/test_core.py::test_subtract PASSED [ 50%]
test Run tests tests/test_core.py::test_divide PASSED [ 75%]
test Run tests tests/test_core.py::test_divide_by_zero_raises PASSED [100%]
test Run tests ===== 4 passed in 0.01s =====
```

|        | Your Mac       | GitHub runner  |
|--------|----------------|----------------|
| OS     | darwin (macOS) | linux (Ubuntu) |
| Python | 3.14.6         | 3.12.14        |
| Result | 4 passed       | 4 passed       |

**That table is the entire value proposition of CI.** Your code is not "works on my machine" anymore — it is independently verified on a clean box, from a fresh clone, with a fresh install.

## Part V — Continuous Deployment

### Step 17 Add the deploy job

We rewrote `ci.yml` to add a second job that only runs if `test` succeeds.

#### COMMAND (FULL FINAL WORKFLOW)

```
cat > .github/workflows/ci.yml << 'EOF'
name: CI/CD

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Check out the code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install the package and dev dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -e ".[dev]"

      - name: Run tests
        run: pytest -v

  deploy:
    needs: test
    if: github.ref == 'refs/heads/main' && github.event_name == 'push'
    runs-on: ubuntu-latest
    steps:
      - name: Check out the code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Build the distribution
        run: |
          python -m pip install --upgrade pip build
          python -m build

      - name: Upload the artifact
        uses: actions/upload-artifact@v4
        with:
          name: calculator-dist
          path: dist/

      - name: Simulate deployment
        run: |
          echo "Deploying calculator to production..."
          ls -lh dist/
          echo "Deploy complete."

EOF
```

#### Two lines carry all the weight:

- `needs: test` — **the gate**. `deploy` will not start until `test` finishes green. If a test fails, deploy never runs at all.
- `if: github.ref == 'refs/heads/main' && github.event_name == 'push'` — build artifacts on real pushes to `main`, but not on pull requests. You want PRs *tested*, not *shipped*.

Back to the kitchen analogy: `needs` is the expediter refusing to pass the plate.

### Step 18 Push and watch the full pipeline

#### COMMAND

```
git add .github/workflows/ci.yml
git commit -m "Add deploy job gated on tests"
git push
gh run watch
```

#### OUTPUT

| STATUS | TITLE                         | WORKFLOW | BRANCH | EVENT | ID          | ELAPSED |
|--------|-------------------------------|----------|--------|-------|-------------|---------|
| ✓      | Add deploy job gated on tests | CI/CD    | main   | push  | 33716983092 | 34s     |
| ✓      | Add CI workflow               | CI/CD    | main   | push  | 33716664512 | 15s     |

**34 seconds** versus 15s for the test-only run — roughly double, which is the deploy job doing its work ( `python -m build` has to construct a wheel and a source tarball).

### Step 19 Confirm both jobs ran in sequence

#### COMMAND

```
gh run view 33716983092
```

#### OUTPUT

```
✓ main CI/CD · 33716983092
Triggered via push about 1 minute ago

JOBS
✓ test in 16s (ID 100528130212)
✓ deploy in 12s (ID 100528185212)

ANNOTATIONS
! Node.js 20 is deprecated. The following actions target Node.js 20 but are being
  forced to run on Node.js 24: actions/checkout@v4, actions/setup-python@v5

ARTIFACTS
calculator-dist
```

#### COMMAND — DOWNLOAD WHAT THE PIPELINE BUILT

```
gh run download 33716983092 --dir /tmp/artifact
ls -lh /tmp/artifact/calculator-dist/
```

#### OUTPUT

```
total 16
-rw-r--r--  1 paulpaul  wheel   1.5K Sep  2 23:59 calculator-0.1.0-py3-none-any.whl
-rw-r--r--  1 paulpaul  wheel   1.4K Sep  2 23:59 calculator-0.1.0.tar.gz
```

Two real, shippable packages — a `.whl` (installable wheel) and a `.tar.gz` (source distribution) — produced by a machine, not by you.

**On the Node.js 20 deprecation warning:** it is noise for now. The actions still run; GitHub is nudging maintainers toward newer versions. Safe to ignore in this demo.

## Part VI — Proving the Gate Works

A pipeline that has only ever been green has not proven anything. **The gate matters only if it can actually stop something.** So we broke it deliberately.

### Step 20 Break it on purpose

#### COMMAND

```
sed -i '' 's/    return a - b/    return a + b # BUG: should be minus/' src/calculator/core.py
pytest -v
```

**macOS gotcha:** note the empty `''` after `-i`. That is a BSD `sed` quirk on macOS — on Linux it would just be `sed -i`. Small, but it will bite you when copying scripts between the two.

#### OUTPUT

```
collected 4 items

tests/test_core.py::test_add PASSED                [ 25%]
tests/test_core.py::test_subtract FAILED            [ 50%]
tests/test_core.py::test_divide PASSED              [ 75%]
tests/test_core.py::test_divide_by_zero_raises PASSED [100%]

===== FAILURES =====
_____ test_subtract _____

    def test_subtract():
>     assert subtract(10, 4) == 6
E       assert 14 == 6
E         + where 14 = subtract(10, 4)

tests/test_core.py:12: AssertionError
===== short test summary info =====
FAILED tests/test_core.py::test_subtract - assert 14 == 6
===== 1 failed, 3 passed in 0.02s =====
```

`assert 14 == 6` — pytest tells you the *actual value*, not just that something failed. That introspection is why the bare `assert` is enough.

### Step 21 Push the broken code

#### COMMAND

```
git add src/calculator/core.py
git commit -m "Introduce bug in subtract (intentional)"
git push
sleep 45
gh run list --limit 3
```

#### OUTPUT

| STATUS | TITLE                                   | WORKFLOW | BRANCH | EVENT | ID          | ELAPSED |
|--------|-----------------------------------------|----------|--------|-------|-------------|---------|
| X      | Introduce bug in subtract (intentional) | CI/CD    | main   | push  | 33717196193 | 12s     |
| ✓      | Add deploy job gated on tests           | CI/CD    | main   | push  | 33716983092 | 34s     |
| ✓      | Add CI workflow                         | CI/CD    | main   | push  | 33716664512 | 15s     |

**X** — the pipeline caught it. Note the elapsed time: **12s versus 34s** for the green run. It bailed early because deploy never ran.

### Step 22 Confirm deploy was *skipped*, not failed

#### COMMAND

```
gh run view 33717196193
```

#### OUTPUT

```

X main CI/CD · 33717196193
Triggered via push about 1 minute ago

JOBS
X test in 8s (ID 100528764471)
  ✓ Set up job
  ✓ Check out the code
  ✓ Set up Python
  ✓ Install the package and dev dependencies
X Run tests
  - Post Set up Python
  ✓ Post Check out the code
  ✓ Complete job
- deploy (ID 100528802066)

ANNOTATIONS
X Process completed with exit code 1.
test: .github#38

```

**The most important line in this whole document:** - `deploy` — a **dash**, not an X. Deploy was *skipped*. It did not run and fail; it never started. And there is **no ARTIFACTS section at all** — the broken wheel was never built, so there was nothing that could accidentally get shipped.

Also worth noting: `Run tests` failed, but `Post Check out the code` and `Complete job` still ran. **Cleanup steps always execute** so the runner does not leak state between jobs.

## Step 23 Fix the bug

### COMMAND

```

sed -i 's/    return a + b # BUG: should be minus/    return a - b/' src/calculator/core.py
pytest -v

```

### OUTPUT

```

collected 4 items

tests/test_core.py::test_add PASSED                [ 25%]
tests/test_core.py::test_subtract PASSED           [ 50%]
tests/test_core.py::test_divide PASSED             [ 75%]
tests/test_core.py::test_divide_by_zero_raises PASSED [100%]

===== 4 passed in 0.00s =====

```

Verify locally *before* pushing — a habit that saves you a red run.

## Step 24 Push the fix

### COMMAND

```

git add src/calculator/core.py
git commit -m "Fix subtract bug"
git push
sleep 50
gh run list --limit 4

```

### OUTPUT

| STATUS | TITLE                                   | WORKFLOW | BRANCH | EVENT | ID          | ELAPSED |
|--------|-----------------------------------------|----------|--------|-------|-------------|---------|
| ✓      | Fix subtract bug                        | CI/CD    | main   | push  | 33717366987 | 34s     |
| X      | Introduce bug in subtract (intentional) | CI/CD    | main   | push  | 33717196193 | 12s     |
| ✓      | Add deploy job gated on tests           | CI/CD    | main   | push  | 33716983092 | 34s     |
| ✓      | Add CI workflow                         | CI/CD    | main   | push  | 33716664512 | 15s     |

The full arc in one screen: ✓ ✓ **X** ✓. And the timing tells the story on its own: **34s when it deploys, 12s when it bails**.

# Recap and Reference

## The mechanism, in one sentence

Push → GitHub spins up a clean Ubuntu VM → installs from scratch → runs tests → and only on success builds and ships an artifact.

## Three concepts to carry forward

| Concept         | Where you saw it                                                                |
|-----------------|---------------------------------------------------------------------------------|
| Reproducibility | Passed on macOS/3.14 and Linux/3.12 — not "works on my machine"                 |
| The gate        | <code>needs: test</code> meant deploy was <i>skipped</i> , never even attempted |
| Fast feedback   | 12 seconds from bad push to red X                                               |

## Command cheat sheet

| Command                                                                     | What it does                                         |
|-----------------------------------------------------------------------------|------------------------------------------------------|
| <code>gh auth login</code>                                                  | Connect the CLI to your GitHub account               |
| <code>gh auth status</code>                                                 | Check who you are logged in as                       |
| <code>git init -b main</code>                                               | New repo with <code>main</code> as the first branch  |
| <code>git status --short</code>                                             | Compact view — verify ignore rules before committing |
| <code>git log --oneline</code>                                              | History, one commit per line with short hash         |
| <code>python3 -m venv .venv</code>                                          | Create an isolated environment                       |
| <code>source .venv/bin/activate</code>                                      | Activate it ( <code>deactivate</code> to exit)       |
| <code>pip install -e ".[dev]"</code>                                        | Editable install + dev extras                        |
| <code>pytest -v</code>                                                      | Run tests, one named line each                       |
| <code>gh repo create NAME --public --source=. --remote=origin --push</code> | Create remote + push in one shot                     |
| <code>gh run list --limit N</code>                                          | Recent pipeline runs with status and IDs             |
| <code>gh run watch</code>                                                   | Stream an in-progress run live                       |
| <code>gh run view ID</code>                                                 | Job-level summary (which jobs ran, skipped, failed)  |
| <code>gh run view ID --log</code>                                           | Full logs (ID required when piping)                  |
| <code>gh run view ID --log-failed</code>                                    | Logs for failed steps only                           |
| <code>gh run download ID --dir DIR</code>                                   | Pull down build artifacts                            |

## Gotchas encountered (worth remembering)

| Gotcha                                  | Detail                                                                             |
|-----------------------------------------|------------------------------------------------------------------------------------|
| Workflow path must be exact             | <code>.github/workflows/</code> — a typo means silent no-op, no error              |
| Branch name must match                  | <code>on: push: branches: [main]</code> must match your actual branch              |
| <code>gh run watch</code> finds nothing | The run already finished. GitHub is often faster than you can type.                |
| Piping <code>gh run view</code> fails   | Pass the run ID explicitly; the interactive picker cannot work in a pipe           |
| macOS <code>sed -i</code>               | BSD sed needs <code>sed -i ''</code> ; GNU/Linux sed uses bare <code>sed -i</code> |
| Node.js 20 deprecation warning          | Cosmetic for now — actions still run on Node 24                                    |

## Cleanup

```
deactivate    # exit the virtual environment when done
```

Repository: <https://github.com/ftk1000/ci-cd-demo> — the Actions tab shows the job dependency as an actual arrow in its graph view.

## Where to go next (tomorrow's session)

---

In rough order of value:

| # | Extension                 | Why it is worth it                                                                                                                      |
|---|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| 1 | <b>Matrix builds</b>      | Test against Python 3.9 through 3.13 in parallel with a <code>strategy.matrix</code> block. Six lines of YAML, six times the coverage.  |
| 2 | <b>Linting</b>            | Add <code>ruff check</code> as a step before tests. Catches style and bug-prone patterns before the tests even run.                     |
| 3 | <b>Coverage reporting</b> | <code>pytest --cov</code> plus a badge in your README.                                                                                  |
| 4 | <b>Branch protection</b>  | Make GitHub <i>refuse</i> merges to <code>main</code> when CI is red. Right now the gate stops deploys; this stops the merge itself.    |
| 5 | <b>Real deployment</b>    | Swap the <code>echo</code> for <code>twine upload</code> to PyPI, or a <code>docker push</code> , using GitHub Secrets for credentials. |

## Questions to test your own understanding

---

1. Why did we use `git add .github/workflows/ci.yml` instead of `git add .` in Step 13?
2. What would happen if you renamed the branch to `master` but left the workflow unchanged?
3. Why is `deploy` shown with a dash rather than an X in Step 22, and why does that distinction matter?
4. What breaks if you delete `where = ["src"]` from `pyproject.toml`?
5. The deploy job checks out the code again. Why does it not inherit the checkout from the `test` job?
6. Why does the `if:` condition on deploy exclude pull requests?